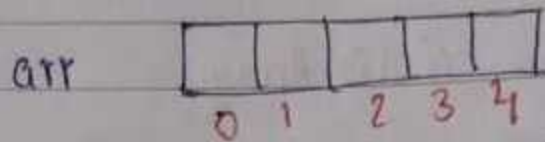
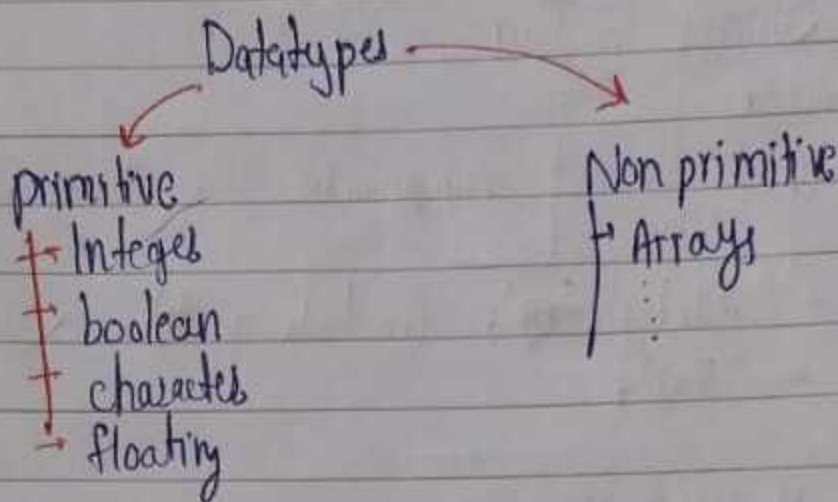


* How Arrays are stored in memory
`int [] arr = new int[5];`



`System.out.println(arr[2]);`
 Random Access



`int x = 4;`

`x = 4`

means 'x' directly hold value 4

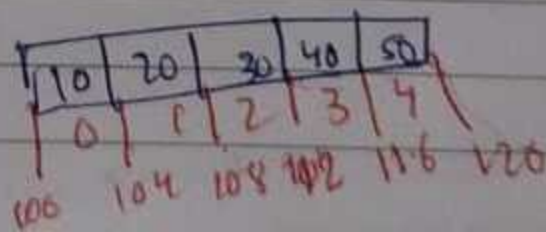
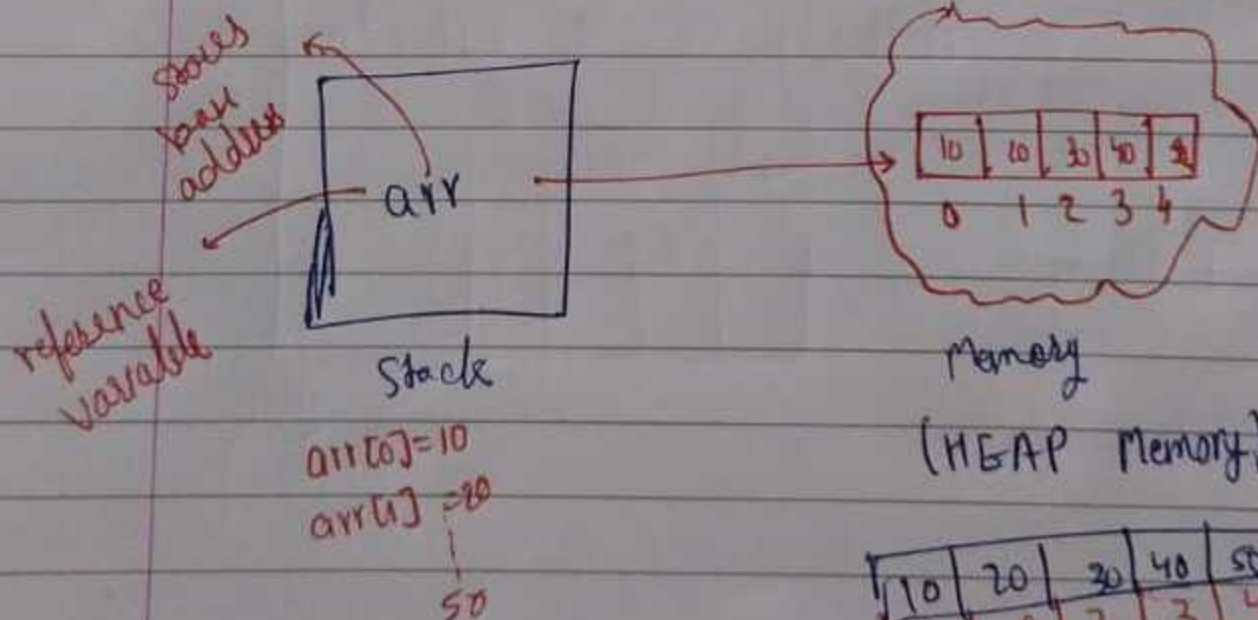
Stack memory

Primitive Data types

`int [] arr = new int[5];`

arr: it does not directly holds the array

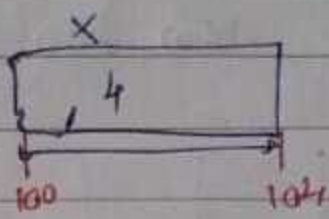
→ It is a reference to an array.



int = 4 bytes

x = 4:

4 bytes



System.out.println(arr[3])

$$\text{arr}[3] = 100 + (4 \times 3) = 112$$

$$\text{arr}[4] = 100 + (4 \times 4) = 116$$

$$\text{arr}[i] = \begin{matrix} \text{Base} \\ \text{Address} \end{matrix} + \left(\begin{matrix} \text{Data type} \\ \text{Size} \end{matrix} * \text{index} \right) \quad \text{K/Imp}$$

Random Access for an array

Boolean → acc to official Java docs

T F

No fix size

Reason:

Boolean

T F
0 1

Hotspot (or called openJDK) → Boolean →

~~1 bit~~ 1 byte

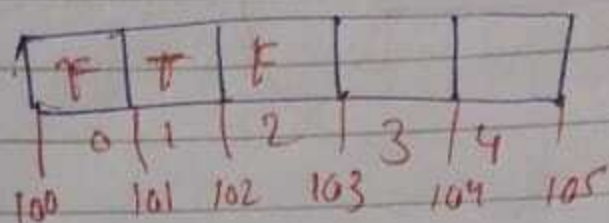
float

byte

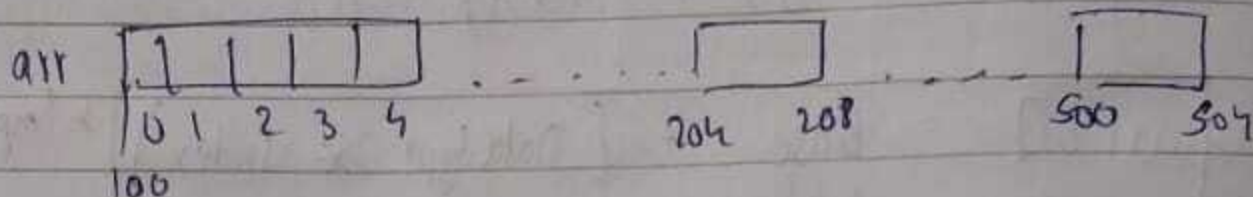
For better CPU optimisation

Boolean ⇒ 1 Byte

`boolean[] arr = new boolean[5]`



$$\begin{aligned} \text{arr}[2] &= 100 + (1 * 2) \\ &= 102 \end{aligned}$$



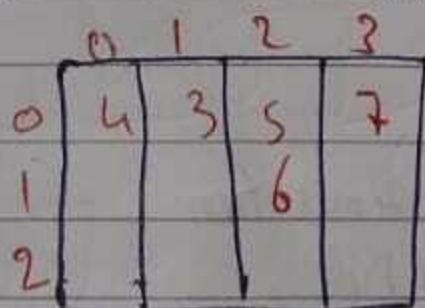
`arr[100]`

$$\begin{aligned} &\rightarrow 100 + (4 * 100) \\ &= 500 \end{aligned}$$

Array Index Out of Bound
 if $(\text{index} < 0 \parallel \text{index} > \text{arr.length})$ {
 throw **ArrayIndexOutOfBoundsException**;
 }

2-D Arrays

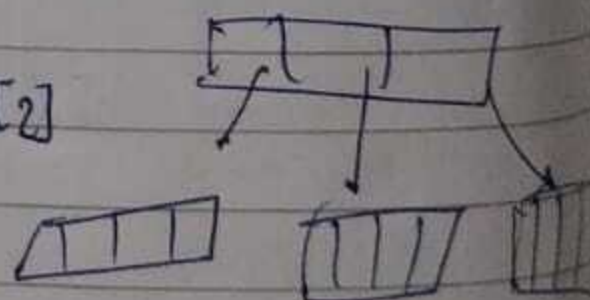
`int[][] arr = new int[3][4];`



array
of arrays

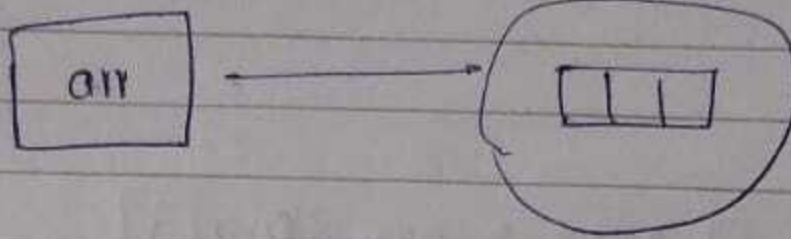
→ we're an array of size 3, where each element is itself an array of size = 4.

`arr[1][2]`

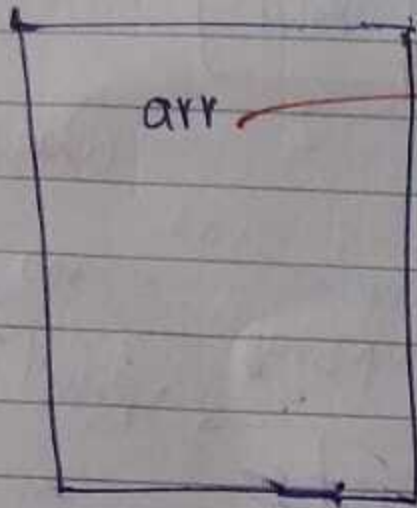


arr [arr[0]] [arr[1]] [arr[2]]
0 1 2

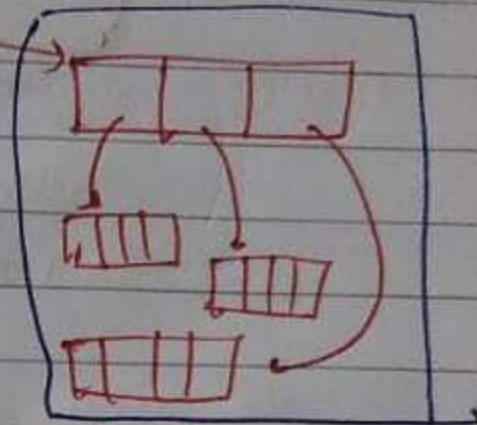
int[] IDArr = new int[3]



reference: arr[0] arr[1] arr[2]

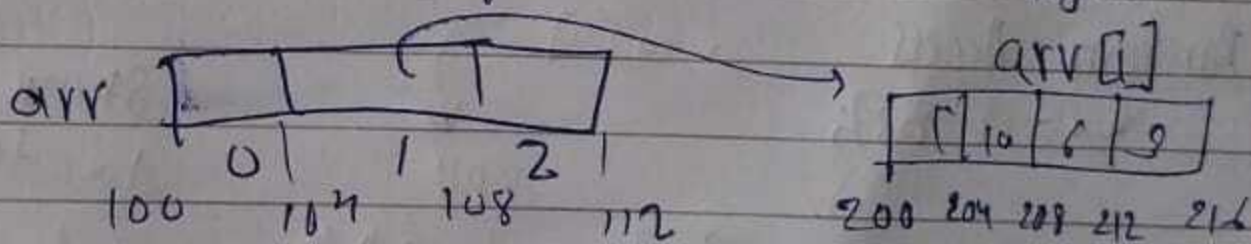


Stack
memory



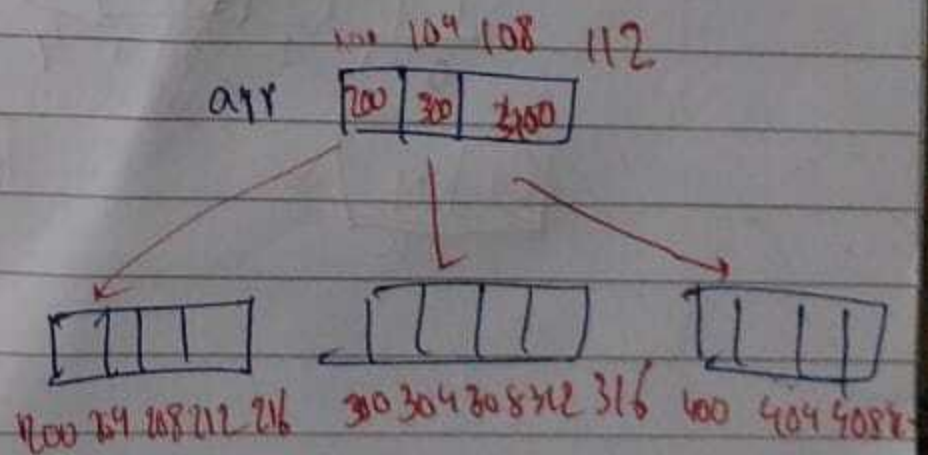
Heap
memory

arr[i][j] — reference variable — 4 Bytes



$$\text{row} = 100 + (4 * 1) \\ = 104$$

$$\text{col} \rightarrow 200 + (4 * 2) \\ = 208$$



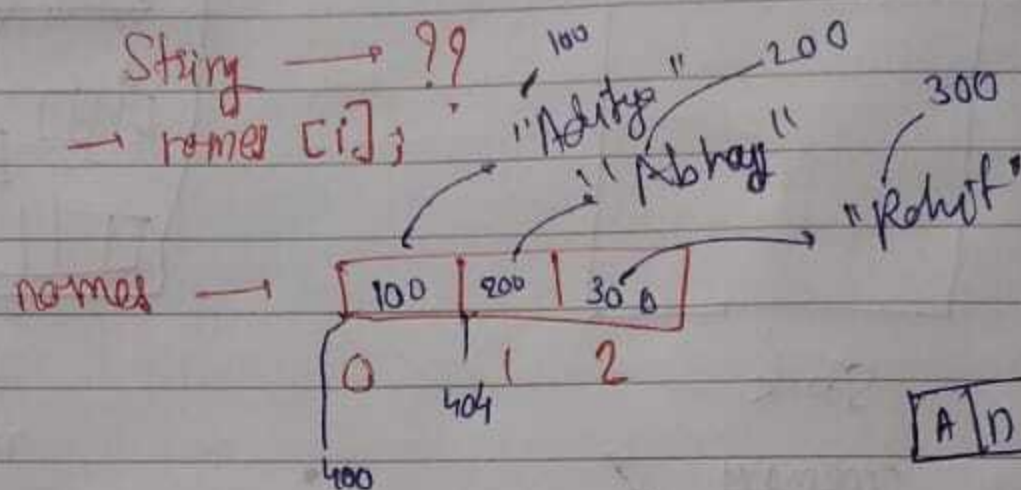
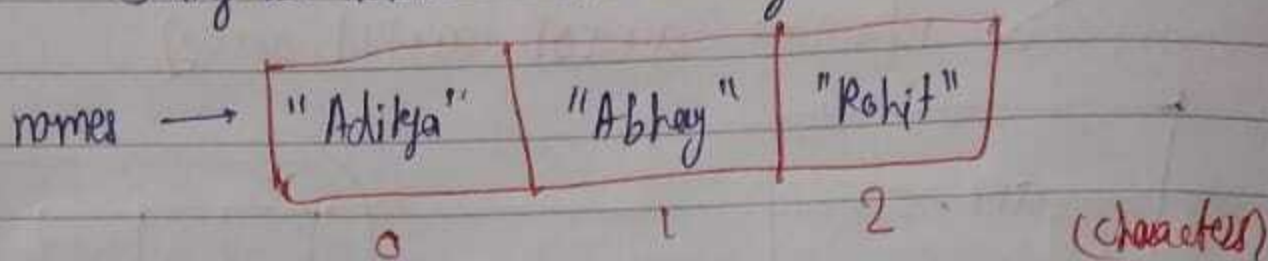
$$\text{arr}[i][j] \rightarrow \left[\begin{array}{l} \text{Base} \\ \text{value} \end{array} \left[\begin{array}{l} \text{address} \\ \text{address} \end{array} \right] + (4 * (\text{index}[i])) \right] - X$$

$$\text{col} \rightarrow [X + (4 * j)]$$

Box Address
2nd Array

String :

String [] names = new String[3]



A | n | i | t | y | a

$$= 400 + (4 * 1)$$

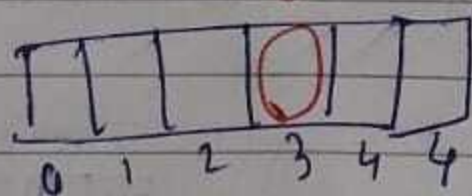
$$= 404$$

ADITYA

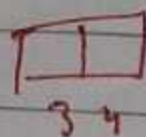
JDK-9

Random Access

arr[3];



Caching



CPU

8 byte

Strings used to
character array but

upto JDK-9

now they are more
complex which we'll
study later in more
complex topic